

Pricer Multi-Produits

Modélisation et Pricing de produits structurés

Documentation

Technique & Fonctionnelle

Formation	Master 272 – Ingénierie Économique et Financière
Université	Paris Dauphine
Module	Modélisation et Pricing de produits structurés
Année	2025–2026
Auteurs	N. LIESSE, H. SERICOLA, M. JEHANNIN

Table des matières

Introduction	3
1 Architecture générale	4
1.1 Logique de séparation des produits	4
2 Hiérarchie des classes	6
2.1 Rôle de chaque classe abstraite	6
2.2 Pattern composite : exemple du CouponBond	6
3 Modèles de taux	8
3.1 Courbe de taux (Nelson-Siegel-Svensson)	8
3.2 Modèle de Vasicek	8
3.3 Modèle CIR	8
3.4 Modèle Hull-White 1F	9
4 Modèles de volatilité	10
4.1 Surface de volatilité implicite	10
4.2 Modèle de Heston	10
5 Pricing par produit	11
5.1 Produits de taux	11
5.2 Produits equity	11
5.3 Notes structurées (mixte)	12
6 Métriques de risque	13
6.1 Sensibilités classiques	13
6.2 Duration et convexité	13
6.3 Bucketing par projection barycentrique	13
6.4 Bucketing par moneyness	13
6.5 Attribution de P&L	14
7 Calibration des modèles	15
8 Choix de design	15
8.1 Pourquoi un fichier par classe	15
8.2 Pourquoi le pattern composite	15
8.3 Robustesse numérique	16
9 Installation et lancement	17
9.1 Prérequis	17
9.2 Installation	17
9.3 Lancement	17
10 Présentation des onglets	18
10.1 Onglet 1 – Marché	18
10.2 Onglet 2 – Pricing interactif	18
10.3 Onglet 3 – Portefeuilles de démonstration	18
10.4 Onglet 4 – Inventaire	19
10.5 Onglet 5 – Risques	19
10.6 Onglet 6 – Calibration	19

11 Workflows utilisateur	20
11.1 Workflow 1 – Pricer un Reverse Convertible	20
11.2 Workflow 2 – Analyser le risque d’un portefeuille	20
11.3 Workflow 3 – Recalibrer Heston	20
12 Format attendu d’Inventaire.xlsx	22
12.1 Feuille Swap	22
12.2 Feuille Options	22
12.3 Feuille Autocall	22
12.4 Feuille Notes structurées	22
13 Codes SSPA supportés	23

Introduction

Ce document constitue la documentation accompagnant le projet *Pricer Multi-Produits*, développé dans le cadre de l'évaluation du cours de modélisation et pricing de produits structurés, enseigné par L. DAVOUST.

L'application répond aux exigences du sujet :

- pricing de plusieurs produits financiers (taux, equity, hybrides) ;
- calcul de métriques de risque agrégées par piliers de maturité $\times$ strike ;
- prise en charge d'un inventaire de portefeuille au format Excel ;
- calibration des modèles utilisés (Vasicek, CIR, Hull-White, Heston) ;
- tableau de bord interactif Streamlit.

L'architecture est orientée objet et conçue pour être *scalable* : ajouter un nouveau produit ou un nouveau modèle se fait en créant un seul fichier Python qui hérite d'une classe abstraite existante.

La documentation est structurée en deux parties :

- **Partie I – Documentation technique** : à destination de l'utilisateur qui souhaite comprendre l'architecture du code, les formules mathématiques et modèles utilisés.
- **Partie II – Documentation fonctionnelle** : à destination de l'utilisateur qui souhaite installer, lancer et utiliser l'application pour pricer des produits ou analyser un portefeuille.

Partie I

Documentation technique

1. Architecture générale

L'arborescence du projet sépare les **modèles**, les **produits** et les **portefeuilles**. Les données de marché sont isolées dans un dossier dédié.

```
pricer/
|-- models/
|   |-- rates/           # Courbe NSS + modeles stochastiques
|   |   |-- rate_curve.py
|   |   |-- stochastic_rate_model.py  (ABC)
|   |   |-- vasicek_model.py
|   |   |-- cir_model.py
|   |   '-- hull_white_model.py
|   '-- volatility/
|       |-- black_scholes.py          (utilitaires)
|       |-- implied_vol_surface.py
|       '-- heston_model.py
|
|-- products/
|   |-- base/                (7 classes abstraites)
|   |-- bond/                (11 produits de taux)
|   |-- equity/              (11 produits equity)
|   '-- mixte/               (5 notes structurees)
|
|-- portfolio/
|   |-- portfolio.py
|   |-- inventory_loader.py
|   '-- default_portfolios.py
|
'-- data/
    |-- loader.py
    |-- rate_curves.parquet
    |-- options.csv
    '-- Inventaire.xlsx

app.py          (interface Streamlit)
requirements.txt
```

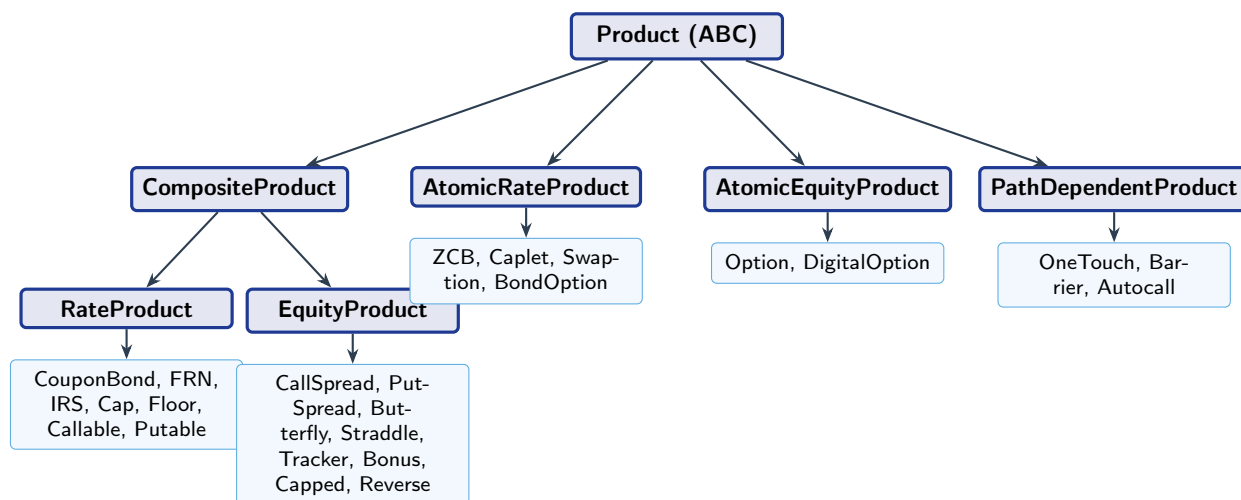
1.1. Logique de séparation des produits

Les produits sont répartis en trois branches selon les **sous-jacents** dont ils sont composés :

Branche	Critère	Facteurs
bond/	dépend uniquement de la courbe de taux	$r(T)$
equity/	dépend d'un sous-jacent equity, taux constant	S, σ
mixte/	combine une jambe taux et une jambe equity	$r(T), S, \sigma$

2. Hiérarchie des classes

Toutes les classes de produits dérivent d'une classe abstraite commune **Product**. Ce pattern permet de construire des produits complexes comme sommes pondérées de produits plus simples.



2.1. Rôle de chaque classe abstraite

Product

Interface commune. Impose les méthodes `price()`, `greeks()` et `to_dict()`.

CompositeProduct

Produit composé d'autres produits. Implémente automatiquement $\text{price} = \sum_i w_i \cdot \text{leg}_i.\text{price}$ et l'agrégation des greeks par sommation pondérée.

AtomicRateProduct

Brique élémentaire de la famille taux. Garantit que `greeks()` contient au minimum `price` et `dv01`.

AtomicEquityProduct

Brique élémentaire equity. Garantit `delta`, `gamma`, `vega`, `theta`, `rho`.

PathDependentProduct

Produit path-dependent (option barrière, autocall). Calcule les greeks par bumping sur le spot.

RateProduct, EquityProduct

Tags de famille pour les composites, permettant de distinguer un swap (taux) d'un call spread (equity).

2.2. Pattern composite : exemple du CouponBond

L'illustration la plus parlante du pattern composite est la décomposition d'une obligation à coupons en somme de zéro-coupons :

$$B = \sum_{i=1}^N \frac{c}{m} \cdot e^{-r(t_i) \cdot t_i} + FV \cdot e^{-r(N) \cdot N}$$

Concrètement, le constructeur ajoute N jambes `ZeroCouponBond` (une par coupon) plus une dernière pour le remboursement du nominal. Le prix et les greeks sont alors automatiquement obtenus par sommation, sans aucune ligne de code de pricing à écrire dans `CouponBond` :

L'arbre de composition complet pour les produits de taux est le suivant :

```
ZeroCouponBond      <-- atomique (brique de base)
|
+-- CouponBond      = somme de ZCB
+-- FloatingRateBond = somme de ZCB(forward)
+-- InterestRateSwap = CouponBond - FloatingRateBond
+-- CallableBond     = CouponBond - BondOption(call)
+-- PuttableBond     = CouponBond + BondOption(put)
```


3. Modèles de taux

3.1. Courbe de taux (Nelson-Siegel-Svensson)

La courbe zéro-coupon est calibrée par la formule paramétrique de Svensson, extension de Nelson-Siegel à 6 paramètres :

$$y(t) = \beta_0 + \beta_1 \frac{1 - e^{-\lambda_1 t}}{\lambda_1 t} + \beta_2 \left(\frac{1 - e^{-\lambda_1 t}}{\lambda_1 t} - e^{-\lambda_1 t} \right) + \beta_3 \left(\frac{1 - e^{-\lambda_2 t}}{\lambda_2 t} - e^{-\lambda_2 t} \right)$$

Interprétation :

- β_0 : taux asymptotique long terme ;
- β_1 : pente initiale (court terme) ;
- β_2, β_3 : courbures (court et moyen terme) ;
- λ_1, λ_2 : vitesses de décroissance exponentielle.

Calibration : minimisation OLS des écarts carrés entre les taux modélisés et les taux observés (3 initialisations distinctes pour éviter les minima locaux), via `scipy.optimize.minimize` avec algorithme L-BFGS-B.

3.2. Modèle de Vasicek

Le taux court suit un processus d'Ornstein-Uhlenbeck :

$$dr_t = a(k - r_t) dt + \sigma dW_t$$

avec a vitesse de retour à la moyenne, k taux long terme, σ volatilité.

Prix d'un ZCB (formule fermée) :

$$B(t) = A(t) \cdot e^{-\frac{\sigma_b}{\sigma} r_0}, \quad \sigma_b = \sigma \cdot \frac{1 - e^{-at}}{a}$$

$$A(t) = \exp \left[\left(\frac{\sigma_b}{\sigma} - t \right) \left(k - \frac{\sigma^2}{2a^2} \right) - \frac{\sigma_b^2}{4a} \right]$$

Calibration : OLS sur la discrétisation $r_{t+\Delta t} = (1 - a\Delta t)r_t + ak\Delta t + \sigma\sqrt{\Delta t}\epsilon$.

3.3. Modèle CIR

Variante de Vasicek avec volatilité dépendante du taux courant :

$$dr_t = a(k - r_t) dt + \sigma\sqrt{r_t} dW_t$$

Avantages : exclut les taux négatifs sous condition de Feller $2ak > \sigma^2$.

Prix d'un ZCB (cours, slide 42) :

$$B(t) = \Phi(t) \cdot e^{-r\Gamma(t)}, \quad \gamma = \sqrt{a^2 + 2\sigma^2}$$

$$\Gamma(t) = \frac{2(e^{\gamma t} - 1)}{(\gamma + a)(e^{\gamma t} - 1) + 2\gamma}$$

$$\Phi(t) = \left[\frac{2\gamma e^{(a+\gamma)t/2}}{(\gamma + a)(e^{\gamma t} - 1) + 2\gamma} \right]^{2ak/\sigma^2}$$

Calibration : maximum de vraisemblance par évolution différentielle (`scipy.optimize.differential_evolution`) avec contrainte de Feller.

3.4. Modèle Hull-White 1F

Extension exacte à la courbe initiale :

$$dr_t = (\theta(t) - ar_t) dt + \sigma dW_t$$

avec

$$\theta(t) = af_t(0) + f'_t(0) + \frac{\sigma^2}{2a} (1 - e^{-2at})$$

où $f_t(0)$ est le taux forward instantané vu en $t = 0$.

Prix d'un ZCB : formule fermée ajustée à la courbe initiale pour garantir l'arbitrage parfait sur les ZCB de marché.

4. Modèles de volatilité

4.1. Surface de volatilité implicite

Pour chaque option du marché, la volatilité implicite est obtenue par inversion de la formule Black-Scholes via Newton-Raphson :

$$\sigma_{n+1} = \sigma_n - \frac{\text{BS}(\sigma_n) - P_{\text{marché}}}{\nu(\sigma_n)}$$

où $\nu = \frac{\partial \text{BS}}{\partial \sigma}$ est le vega.

Convention de cotation :

- puts pour $K < S$ (out-of-the-money en put = in-the-money en call) ;
- calls pour $K > S$.

La surface est ensuite interpolée par spline bicubique sur la grille (strike, maturité) avec `scipy.interpolate.RectBivariateSpline`.

4.2. Modèle de Heston

Volatilité stochastique avec corrélation entre spot et variance :

$$\begin{aligned} dS_t &= rS_t dt + \sqrt{v_t} S_t dW_t^S \\ dv_t &= \kappa(\theta - v_t) dt + \zeta \sqrt{v_t} dW_t^v \\ d\langle W^S, W^v \rangle_t &= \rho dt \end{aligned}$$

Pricing par Carr-Madan FFT : le prix du call s'obtient par transformée de Fourier rapide de la fonction caractéristique de $\log S_T$:

$$C(K) = \frac{e^{-\alpha \log K}}{\pi} \int_0^\infty e^{-iv \log K} \psi(v) dv$$

avec $\psi(v)$ donnée par la fonction caractéristique de Heston. La discrétisation de l'intégrale par règle de Simpson permet une seule FFT pour calculer le prix sur toute une grille de strikes.

Pricing exotique par Monte Carlo : schéma Euler à variance tronquée (*full truncation*) pour garantir $v_t \geq 0$.

Calibration : double passe *differential evolution* (global) puis L-BFGS-B (local) sur l'écart entre vol implicite Heston et vol implicite marché.

5. Pricing par produit

5.1. Produits de taux

Produit	Méthode	Formule / Décomposition
ZCB	analytique	$B = N \cdot e^{-r(T)T}$
CouponBond	composite	$\sum_i \text{ZCB}(c_i, t_i) + \text{ZCB}(N, T)$
FRN	composite	coupons forward $C_t = (\alpha I_t + m)D_t$
IRS	composite	$\pm(\text{Bond fixe} - \text{FRN})$
Caplet/Floorlet	analytique	Black 76 sur le forward
Cap/Floor	composite	$\sum_i \text{Caplet/Floorlet}$
Swaption	analytique	Black 76 sur le swap rate
BondOption	MC HW	Monte Carlo Hull-White
CallableBond	composite	$\text{Bond} - \text{BondOption}(\text{call})$
PutableBond	composite	$\text{Bond} + \text{BondOption}(\text{put})$

5.1.1. Caplet et Cap

Le caplet, brique du cap, est priced par la formule de Black Scholes :

$$\text{Caplet} = P(0, T_1) \cdot \tau \cdot N \cdot [F \mathcal{N}(d_1) - K \mathcal{N}(d_2)]$$

$$d_{1,2} = \frac{\ln(F/K) \pm \frac{1}{2}\sigma^2 T_1}{\sigma \sqrt{T_1}}$$

où F est le forward sur $[T_1, T_2]$ et $\tau = T_2 - T_1$.

Un cap est la somme des caplets sur chaque période de la maturité.

5.1.2. Swaption

Black Scholes appliqué au taux swap :

$$\text{Swaption}^{\text{payer}} = N \cdot A(0) \cdot [S_0 \mathcal{N}(d_1) - K \mathcal{N}(d_2)]$$

avec $A(0) = \sum_i P(0, T_i) \tau_i$ l'annuité du swap et $S_0 = \frac{P(0, T_{\text{start}}) - P(0, T_{\text{end}})}{A(0)}$ le swap rate à l'origine.

5.2. Produits equity

5.2.1. Option européenne (Black-Scholes-Merton)

$$C = S e^{-qT} \mathcal{N}(d_1) - K e^{-rT} \mathcal{N}(d_2)$$

$$d_{1,2} = \frac{\ln(S/K) + (r - q \pm \frac{1}{2}\sigma^2)T}{\sigma \sqrt{T}}$$

Si une surface de vol implicite est fournie, σ est extrait par interpolation au strike K et à la maturité T . Si un modèle Heston est fourni, le pricing passe par Carr-Madan FFT.

Produit	Méthode	Formule / Décomposition
Option vanille	analytique	BSM ou Heston Fourier
DigitalOption	analytique	cash-or-nothing : $P \cdot e^{-rT} \mathcal{N}(d_2)$
CallSpread	composite	$\text{Call}(K_1) - \text{Call}(K_2)$
PutSpread	composite	$\text{Put}(K_2) - \text{Put}(K_1)$
Butterfly	composite	$+1 \text{ Call}(K_1) - 2 \text{ Call}(K_2) + 1 \text{ Call}(K_3)$
Straddle	composite	$\text{Call} + \text{Put}$ (même K)
Strangle	composite	$\text{Call}(K_2) + \text{Put}(K_1)$
Strip	composite	$+1 \text{ Call} + 2 \text{ Put}$
Strap	composite	$+2 \text{ Call} + 1 \text{ Put}$
OneTouch	analytique	formule de Reiner-Rubinstein
BarrierOption (KO/KI)	MC	Monte Carlo Heston ou GBM

5.2.2. Option à barrière

Pricing Monte Carlo sur trajectoires GBM ou Heston. Quatre types : down-and-out, up-and-out, down-and-in, up-and-in.

KO : payoff vanille si la barrière n'est pas touchée, rebate sinon

KI : payoff vanille si la barrière est touchée, rebate sinon

5.3. Notes structurées (mixte)

Code SSPA	Nom	Réplication
1100	Tracker Certificate	$\text{participation} \times \text{Call}(K \approx 0)$
1130	Bonus Certificate	$\text{Call}(K = 0) + \text{Put}(K_{\text{bonus}}) - \text{Put-KI}$
1220	Capped Capital Protection	$\text{ZCB} + \text{participation} \times \text{CallSpread}$
1320	Reverse Convertible	$\text{CouponBond} - \text{Put-KI}$

Autocall : Monte Carlo avec observations périodiques. À chaque date d'observation t_i , si $S(t_i) \geq B_i \cdot K$, le produit est rappelé avec versement de $1 + i \cdot c$. À maturité, si $S(T) \geq B_{\text{final}} \cdot K$, versement du nominal ; sinon, perte en capital proportionnelle.

6. Métriques de risque

6.1. Sensibilités classiques

Pour chaque produit, les greeks sont calculés analytiquement ou par bumping (path-dependent) :

Greek	Définition	Famille
Δ	$\partial V / \partial S$	equity
Γ	$\partial^2 V / \partial S^2$	equity
ν (vega)	$\partial V / \partial \sigma$	equity
Θ	$\partial V / \partial t$	equity
ρ	$\partial V / \partial r$	equity
DV01	$-\partial V / \partial r \cdot 10^{-4}$	taux

6.2. Duration et convexité

Conformément au cours, pour une obligation à coupons :

$$\begin{aligned}
 D_{\text{Macaulay}} &= \frac{1}{B} \sum_{i=1}^N t_i \cdot e^{-yt_i} \cdot F_i \\
 D_{\text{modifiée}} &= \frac{D_{\text{Macaulay}}}{1 + y/m} \\
 \text{DV01} &= D_{\text{modifiée}} \cdot B \cdot 10^{-4} \\
 C &= \frac{1}{B} \sum_{i=1}^N t_i^2 \cdot e^{-yt_i} \cdot F_i
 \end{aligned}$$

L'approximation au second ordre du prix face à un déplacement parallèle de la courbe de taux s'écrit :

$$\Delta P \approx -D_{\text{modifiée}} \cdot B \cdot \Delta r + \frac{1}{2} \cdot C \cdot (\Delta r)^2$$

6.3. Bucketing par projection barycentrique

Les sensibilités au taux sont **réparties sur les piliers de courbe** (1M, 3M, 6M, 1Y, 2Y, 3Y, 5Y, 7Y, 10Y, 15Y, 20Y, 30Y) par projection barycentrique :

Pour un flux à maturité T entre les piliers t_k et t_{k+1} :

$$w_k = \frac{t_{k+1} - T}{t_{k+1} - t_k}, \quad w_{k+1} = \frac{T - t_k}{t_{k+1} - t_k}$$

6.4. Bucketing par moneyness

Pour les produits equity, le risque est ventilé par bucket de moneyness K/S : <70%, 70-85%, 85-95%, ATM, 105-115%, 115-130%, >130%, ce qui permet d'identifier l'exposition au smile de volatilité.

6.5. Attribution de P&L

Pour un mouvement combiné $(\Delta S, \Delta r, \Delta \sigma, \Delta t)$, le P&L se décompose au premier ordre par les greeks :

$$\Delta V \approx \Theta \cdot \Delta t + \Delta \cdot \Delta S + \frac{1}{2} \Gamma \cdot (\Delta S)^2 + \nu \cdot \Delta \sigma + \rho \cdot \Delta r$$

7. Calibration des modèles

Modèle	Méthode	Données
NSS	OLS multi-init	taux zéro de marché
Vasicek	OLS sur Euler	historique court terme (3M)
CIR	MLE (DE)	historique court terme + contrainte Feller
Hull-White	Calage exact	courbe forward instantanée
Surface impl.	Newton-Raphson	prix mid des options
Heston	DE puis L-BFGS-B	vols implicites de la surface

8. Choix de design

8.1. Pourquoi un fichier par classe

Le projet adopte la convention "un fichier Python par classe", ce qui :

- facilite la navigation (recherche immédiate par nom de fichier)
- rend les imports explicites et auditables
- permet d'ajouter un nouveau produit sans toucher aux fichiers existants
- favorise le travail en équipe (organisation plus simple)

Les `__init__.py` de chaque dossiers re-exportent les classes pour maintenir des imports utilisateur courts (facilite le code, sa relecture et son utilisation) :

```
# Au lieu de :
from pricer.products.bond.coupon_bond import CouponBond
from pricer.products.bond.zero_coupon_bond import ZeroCouponBond

# On écrit :
from pricer.products.bond import CouponBond, ZeroCouponBond
```

8.2. Pourquoi le pattern composite

Le pattern *Composite* évite de réécrire la logique de pricing pour chaque combinaison. Il permet de récupérer créer une combinaison de produit en récupérant directement les produits de base. Par exemple :

- un **Cap** est une somme de **Caplet** (déjà codés) ;
- un **IRS** est la différence d'un **CouponBond** et d'un **FloatingRateBond** (déjà codés) ;
- un **ReverseConvertible** est un **CouponBond** moins une **BarrierOption** (déjà codés).

Ainsi le code de pricing du **ReverseConvertible** se réduit à deux lignes `_add_leg(...)` dans le constructeur, sans aucun calcul mathématique explicite – le code de **CompositeProduct** fait le reste.

8.3. Robustesse numérique

Plusieurs garde-fous ont été mis en place :

- **YTM** : Brent sur $[-20\%, +50\%]$ avec fallback au taux zéro de la courbe si pas de racine ;
- **NSS** : 3 initialisations différentes pour éviter les minima locaux ;
- **Heston** : double passe DE + L-BFGS-B avec contrainte de Feller ;
- **CIR Monte Carlo** : schéma Euler tronqué pour garantir $r \geq 0$;
- **Heston Monte Carlo** : full truncation $v \rightarrow \max(v, 0)$.

Partie II

Documentation fonctionnelle

9. Installation et lancement

9.1. Prérequis

- Python 3.10 ou supérieur
- pip pour la gestion des dépendances

9.2. Installation

```
# 1. Cloner ou extraire le projet

# 2. (Optionnel) Créer un environnement virtuel

# 3. Installer les dépendances
pip install -r requirements.txt
```

Les dépendances installées sont **streamlit**, **pandas**, **numpy**, **scipy**, **plotly**, **openpyxl** et **pyarrow**.

9.3. Lancement

Pour la visualisation des résultats, nous avons utilisé une interface streamlit. Pour afficher cette interface, veuillez rentrer le code suivant dans votre terminal :

```
streamlit run app.py
```

L'application s'ouvre dans le navigateur à l'adresse **http://localhost:8501**. Le chargement initial peut prendre un peu de temps à cause des calculs à effectuer. Les mises à jour suivantes seront plus rapides.

10. Présentation des onglets

L'application est structurée en 6 onglets accessibles depuis la barre principale. La barre latérale gauche permet de configurer les paramètres globaux (sous-jacent, pays de la courbe de taux, modèles, paramètres Monte Carlo).

10.1. Onglet 1 – Marché

Affiche les données de marché courantes :

- **Indicateurs clés** : spot, taux 3M, volatilité ATM 1Y ;
- **Courbe de taux** : visualisation de la courbe NSS calibrée superposée aux taux de marché ;
- **Surface de volatilité implicite** : représentation 3D de la surface implicite (strike $\times$ maturité $\times$ volatilité) ;
- **Smile par maturité** : coupes de la surface pour différentes échéances, mettant en évidence la structure du smile.

10.2. Onglet 2 – Pricing interactif

Pricer en temps réel pour 13 types de produits, regroupés en 4 catégories :

- **Vanilles** : Option Européenne, Call/Put Spread, Butterfly ;
- **Exotiques** : Option à Barrière, Autocall ;
- **Notes structurées** : Tracker (1100), Bonus (1130), Capped Capital (1220), Reverse Convertible (1320) ;
- **Taux** : ZCB, Bond à coupons, Swap de taux.

Pour chaque produit, l'interface affiche :

- le prix calculé selon les paramètres choisis ;
- les greeks pertinents (delta, gamma, vega, theta, rho ou DV01, duration selon la famille) ;
- un graphe de payoff à maturité (pour les produits à payoff explicite).

10.3. Onglet 3 – Portefeuilles de démonstration

Quatre portefeuilles préconstruits illustrent les capacités du pricer :

- **Taux** : ZCB, bonds, FRN, IRS, callable, putable (8 positions) ;
- **Vanilles** : calls, puts, spreads, butterfly (6 positions) ;
- **Exotiques** : barrière, autocall (4 positions) ;
- **Mixte** : combinaison des trois (5 positions).

Pour chaque portefeuille sélectionné, l'application affiche la valeur totale, les greeks agrégés (delta, gamma, vega, DV01) et le détail position par position sous forme de tableau et de graphique en barres.

10.4. Onglet 4 – Inventaire

Cet onglet est le **cœur fonctionnel** de l'application : il charge le fichier `Inventaire.xlsx` et valorise chaque ligne en utilisant les classes du pricer.

Workflow :

1. Le fichier `Inventaire.xlsx` (4 feuilles) est parsé par `InventoryLoader` ;
2. Chaque ligne est convertie en objet produit Python ;
3. Les positions sont regroupées en 4 portefeuilles ;
4. Chaque portefeuille est valorisé sur les données de marché courantes.

L'utilisateur peut consulter les données brutes Excel et les positions valorisées pour chaque feuille.

10.5. Onglet 5 – Risques

Analyse de risque approfondie sur n'importe quel portefeuille (démonstration ou inventaire) :

- **Risque par pilier de maturité** : DV01 et delta ventilés sur les 12 piliers de courbe par projection barycentrique ;
- **Risque par bucket de moneyness** : delta, gamma et vega ventilés sur 7 buckets de K/S ;
- **Attribution de P&L** : décomposition du P&L pour un mouvement de marché donné par l'utilisateur ($\Delta S, \Delta r, \Delta \sigma, \Delta t$).

10.6. Onglet 6 – Calibration

Permet de relancer la calibration de chaque modèle stochastique sur les données historiques :

- **Vasicek** : OLS sur la série historique 3M ;
- **CIR** : différentielle évolutive sous contrainte de Feller ;
- **Heston** : DE puis L-BFGS-B sur la surface implicite, avec diagramme de comparaison vol marché vs vol Heston.

Les paramètres calibrés sont affichés à l'écran et peuvent être réutilisés dans les autres onglets.

11. Workflows utilisateur

11.1. Workflow 1 – Pricer un Reverse Convertible

Objectif : pricer un Reverse Convertible 2 ans sur AAPL avec strike ATM, barrière à 70 % du spot et coupon de 8 %.

1. Dans la *barre latérale* : sélectionner **AAPL** comme sous-jacent et **Heston** comme modèle de volatilité (recommandé pour les options à barrière) ;
2. Aller dans l'onglet *Pricing* ;
3. Sélectionner **Reverse Convertible (1320)** ;
4. Saisir maturité = 2 ans, strike = S_0 , barrière = $0.7 \cdot S_0$, coupon = 8%, nominal = 100 ;
5. Cliquer en dehors du dernier champ pour déclencher le calcul ;
6. Le prix Monte Carlo (5 000 simulations par défaut) s'affiche, ainsi que le coupon, la barrière et le strike rappelés.

Lecture du résultat : si le prix est proche de 100 €, le coupon proposé est cohérent avec la prime de risque demandée par le marché. Si le prix est très inférieur, le coupon est insuffisant pour rémunérer le risque.

11.2. Workflow 2 – Analyser le risque d'un portefeuille

Objectif : comprendre l'exposition aux taux et au spot du portefeuille "Mixte".

1. Onglet *Portefeuilles démo* : sélectionner le portefeuille **Mixte** et vérifier les agrégats (DV01, delta) ;
2. Onglet *Risques* : sélectionner la même source et le même portefeuille ;
3. Lire le *risque par pilier de maturité* : on identifie les zones de la courbe où le DV01 est concentré ;
4. Lire le *risque par bucket de moneyness* : on identifie si le delta et le vega sont concentrés ATM ou OTM ;
5. Tester une *attribution P&L* avec un choc combiné, par exemple : $\Delta S = +5$, $\Delta r = +25$ bp, $\Delta \sigma = -2\%$;
6. Le tableau de décomposition montre la contribution de chaque greek (delta, gamma, vega, rho, theta) au P&L total.

11.3. Workflow 3 – Recalibrer Heston

Objectif : obtenir des paramètres Heston frais sur la surface du jour.

1. Onglet *Calibration* ;
2. Cliquer sur **Calibrer Heston** ;
3. Patienter 10-30 secondes (DE est gourmande) ;
4. Lire les paramètres calibrés : v_0 , κ , θ , ζ , ρ ;

5. Examiner le diagramme de dispersion vol marché vs vol Heston : plus les points sont alignés sur la diagonale, meilleur est le fit ;
6. Les paramètres calibrés sont automatiquement utilisés par tous les pricings exotiques (auto-call, barrière) tant que la session est active.

12. Format attendu d'Inventaire.xlsx

Le fichier Excel doit contenir 4 feuilles avec les colonnes suivantes.

12.1. Feuille Swap

Colonne	Type	Exemple
Date Valorisation	date	2026-02-27
Maturité	date	2031-02-27
Nominal	numérique	1 000 000
Taux fixe	numérique ou vide	0.035
Fréquence fixe	texte	6M

Si **Taux fixe** est vide, la ligne est interprétée comme un *basis swap* (deux jambes variables) et modélisée par un `FloatingRateBond`.

12.2. Feuille Options

Colonne	Type	Exemple
Date Valorisation	date	2026-02-27
Maturité	date	2027-02-27
Quantité	entier	100
Produit	texte	Call Spread, Butterfly, Call, Put
Strike 1	numérique	250
Strike 2	numérique ou vide	280
Strike 3	numérique ou vide	310
Type Barrière	texte ou vide	OUT, IN
Niveau Barrière	numérique ou vide	200

12.3. Feuille Autocall

Une ligne par observation. Le ID **Produit** regroupe les observations d'un même autocall.

Colonne	Type	Exemple
ID Produit	entier	1
Date Référence	date	2026-02-27
Date Valorisation	date	2026-02-27
Date Observation	date	2027-02-27
Niveau de rappel	numérique	1.0
Coupon	numérique	0.06

12.4. Feuille Notes structurées

Colonne	Type	Exemple
Date valorisation	date	2026-02-27
Code produit SSPA	entier	1100, 1130, 1220, 1320
Quantité	entier	100
Sous jacent	texte	AAPL
Taux de participation	numérique ou vide	1.0
Devise taux	texte	USD
Maturité	date	2028-02-27
Barrière 1	numérique ou vide	200
Cap	numérique ou vide	280
Barrière 2	numérique ou vide	250

Code	Nom	Profil de payoff
1100	Tracker Certificate	Réplique linéaire du sous-jacent ($\text{participation} \times S_T$).
1130	Bonus Certificate	Niveau bonus garanti à maturité tant que la barrière n'est pas touchée ; sinon, le porteur reçoit S_T .
1220	Capped Capital Protection	Capital garanti par un ZCB plus participation à la hausse plafonnée par un cap.
1320	Reverse Convertible	Coupon élevé, mais capital exposé à la baisse via un short put barrière.

13. Codes SSPA supportés

SSPA (*Swiss Structured Products Association*) est l'organisme suisse de classification des produits structurés. Ses codes définissent un payoff standardisé indépendamment de l'émetteur, ce qui permet une réplication statique commune à tous les produits du même code.

Fin de la documentation

Master 272 – Ingénierie Économique et Financière – 2026

LIESSE Nathan – SERICOLA Hugo – JEHANNIN Marc